

Fast FPGA-based Emulation for ReRAM-Enabled Deep Neural Network Accelerator

Yongquan Shi, Yongshuai Sun, Jianfei Jiang, Guanghui He, Qin Wang, Naifeng Jing*
Department of Micro/Nano Electronics, Shanghai Jiao Tong University, Shanghai, China

Abstract — Resistive-RAM (ReRAM) based deep neural network (DNN) accelerator has shown great potential to address the memory wall problem for its processing-in-memory (PIM) capacity. However, ReRAM DNN accelerator still faces various challenges in its early architecture design phase due to the unpredictable variability and limitation of the ReRAM device. Software simulation helps but the simulation time is long with detailed ReRAM device model for large-scale DNNs. In this paper, we propose fast FPGA-based emulation for the DNN accelerator of ReRAM device. The emulation sets a primitive DNN accelerator architecture in FPGA and leverages FPGA hardware resources to provide massive parallelism for reducing emulation time. Meanwhile, it is co-designed with runtime software stacks to make the hardware emulation more flexible via instruction compilation and scheduling for different DNN needs. Our experiments show that the emulation can get over 194.7X speedup for large-scale DNNs against NeuroSim as a software simulator for ReRAM DNN accelerator. Therefore, the proposed emulation helps to build better ReRAM accelerators for large DNNs with much higher speed and flexibility.

Keywords — ReRAM, PIM, accelerator, DNN, FPGA-based emulation

I. INTRODUCTION

DNN has been widely adopted in many fields, i.e., image classification, speech recognition and natural language processing. As DNNs are getting more and more complicated, hardware accelerator comes to the rescue but experiences sharply increased computation and a large volume of data movement [1]. To these problems, recently the ReRAM [2] device has emerged as a promising candidate for the accelerator designs owing to the inherited PIM capability. Generally, the ReRAM devices are organized in a crossbar structure [3]. It can perform a matrix vector multiplication (MVM) in analog domain with massive parallelism that can reduce the time complexity from $O(n^2)$ to $O(1)$ in ideal without data movement.

Nevertheless, there still remain many challenges to the ReRAM-based DNN accelerator designs. For example, ReRAM devices suffer from various variations that may lead to inaccurate results during MVM computation [4]. As a result, designers have to carefully evaluate the effect at the hardware level. In addition, different quantization schemes on weight and activation values may be evaluated to balance DNN accuracy and computing efficiency. At the same time, DNN accelerator often equips a large number of crossbars to parallelize the computation. There could be numerous intermediate results at each crossbar to evaluate for correct operations.

This work is partly supported by the National Key R&D Program of China (Grant No. 2018YFA0701500) and Natural Science Foundation of China (Grant No. 61772331). Naifeng Jing is the corresponding author.

In short, many design choices require careful evaluations at each step throughout the ReRAM DNN accelerator design. However, high-level DNN programming framework can be insufficient because it only evaluates algorithm's correctness without considering the underneath hardware characteristics. Software simulation works. But given the large number of intermediate details, the simulation time increases sharply particularly for large-scale DNNs. For example, a software simulator bearing ReRAM behavior and variation model may spend minutes for a dozen of crossbars that mimic hundreds of neurons on a small convolution. When a larger network is investigated, it may require hours or days. When architecture is repeatedly tuned, the time spent on software simulation may become unacceptable.

To accelerate the evaluation, [5] presented an FPGA-based emulator which is several orders of magnitude faster than CPU simulator and permits DNN evaluation with sufficient ReRAM model details. Although feasible, the emulator is only designed for specific application.

In this paper, we propose fast FPGA-based emulation to evaluate the DNN accelerator with ReRAM devices. The emulator should be flexible to cater for different DNNs on ReRAM accelerators. So, we present to emulate one ReRAM crossbar with various effects using multiply-and-accumulate (MAC) units in FPGA, and emulate massive ReRAM resources by dividing them into small pieces that fit in the FPGA using a time-multiplexed manner. To ease hardware complexity, the emulator is co-designed with software stacks. It compiles DNN into an instruction stream with several types of micro-instructions that provides desirable flexibility for the emulation, while the FPGA hardware provides massive parallelism to support micro-instruction execution for shorter emulation. The experiment results show that the emulation gets more than $194.7 \times$ performance gain for complex DNNs against the DNN NeuroSim [6] simulator that in turn permits detailed emulation even with device variation taken into account.

The rest of the paper is organized as follows. Section II and III present the hardware and software designs of the proposed emulator. Then, the experimental results are given in Section IV. Finally, Section V concludes this paper.

II. HARDWARE DESIGN OF FPGA-BASED EMULATOR

A. General ReRAM DNN Accelerator Architecture

The ReRAM-based DNN accelerator is often designed as a hierarchical architecture [7] to reduce unnecessary data movement and make full use of the computing units. For example, ISAAC [8] presents a four-level design of Chip, Tile, IMA and Crossbar to increase the computing throughput, and so do PANTHER [9], PUMA [10] and ERA-LSTM [11].

As shown in Fig. 1, a general ReRAM DNN accelerator may consist of multiple cores, connected to the instruction fetch module via command queues. Each core, mainly

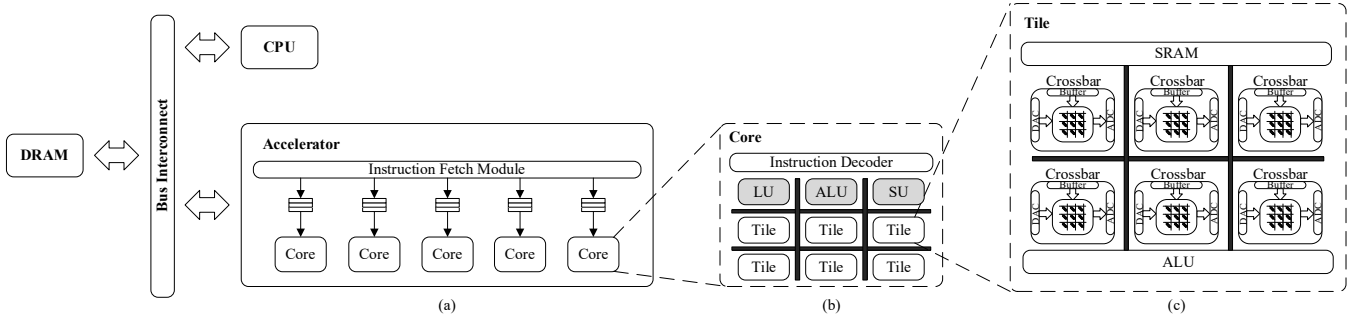


Fig. 1. Architecture of ReRAM DNN accelerator. (a) The accelerator. (b) The core. (c) The tile.

performing network-level operations, consists of multiple tiles, an instruction decoder, a load unit, a store unit and arithmetic and logic units (ALUs) for pooling. And each tile, mainly performing layer-level operations, contains multiple crossbars, an on-chip SRAM and a tensor ALU for activation. Each crossbar consists of $N \times N$ ReRAM cells, a digital-to-analog convertor (DAC), an analog-to-digital convertor (ADC), and a data buffer. MVM operation is mainly executed in crossbars.

During the inference, the CPU may inject the compiled instructions to accelerator, and the instruction fetch module in the accelerator fetches the instructions to each core and decodes to each tile. Then the weight and input values fill the buffer in correspondent crossbars. Finally, it executes MVM, activation, and pooling with final results stored to DRAM.

B. FPGA emulator for ReRAM DNN accelerator

Due to the limited MAC resources in FPGA, it cannot accommodate the ReRAM DNN accelerator as a whole. In this paper, we propose a primitive DNN architecture in FPGA that is reused in a time-multiplexed manner to emulate the whole ReRAM DNN accelerator. Runtime support is also proposed to simplify the hardware design.

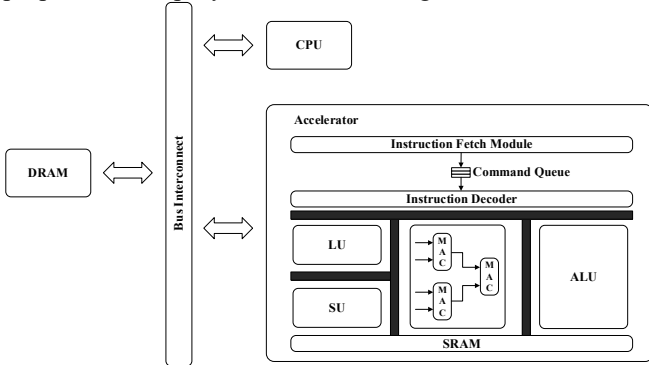


Fig. 2. FPGA primitive architecture to emulate ReRAM DNN accelerator

As illustrated in Fig. 2, the FPGA primitive architecture consists of all the necessary function units in a ReRAM DNN accelerator as shown in Fig. 1. To emulate MVM operation, which is executed in analog domain in a ReRAM crossbar, we replace it with a number of MACs in a tree structure. Specifically, for a $N \times N$ ReRAM crossbar, we have to use at least $N^2/2$ MACs if a single weight value can be embodied in a ReRAM cell. Besides, the primitive architecture also equips an instruction fetch module and a command queue to emulate the instruction sent to the multi-cores. Moreover, it contains a load unit and a store unit to provide data to the

MAC tree unit, and a tensor ALU to execute pooling and activation operations.

Due to the limited MACs in FPGA that are far from full emulation of the parallel MVM in one ReRAM crossbar even with massive crossbars, the MVM is split into small pieces that fit in the FPGA MAC tree and reuse the MAC tree to serially emulate the highly-paralleled MVM operations in one or across a number of ReRAM crossbars. As illustrated in Fig. 3, it is assumed that the inputs and outputs are divided into n subvectors which contain m input scalars per subvector, and weights are divided into $n \times n$ block matrices with $m \times m$ weights. To make full use of every input subvector to multiply the block matrixes, we propose a MVM scheduling on the MAC tree unit. Each input subvector multiplies with n weight block matrixes that are placed in the same input channel with the input subvector and get the partial sum of n output subvectors. It takes $n \times n$ cycles to complete a ReRAM MVM operation, because the emulator can only perform one multiplication of a block matrix and a subvector per cycle.

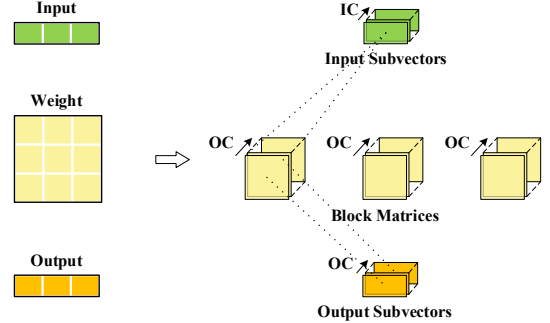


Fig. 3. FPGA MVM emulation

To ease the hardware complexity and a reduced buffer to collect inputs and weights before MVM operation, we further propose adding software stack to support the FPGA primitive architecture for ReRAM DNN emulation in the next section.

III. SOFTWARE/HARDWARE CO-DESIGN OF THE EMULATOR

A. Instruction Set Architecture of ReRAM DNN

The ReRAM DNN accelerators often have instructions customized for ReRAM device and in-memory computing. For example, comparing to traditional accelerators [12], the ReRAM DNN accelerator needs instructions to specify data movement between different hierarchy as in Fig. 1 such as ‘Send’ and ‘Receive’ [13]. Referring the ISA in PANTHER [9], the instructions in ReRAM DNN accelerator mainly consists of four kinds of instructions as listed in Table I.

B. Instruction translation to FPGA Emulator

Obviously, instructions for ReRAM DNN accelerator cannot be executed directly on FPGA primitive hardware. Instead of a dynamic binary translation, we propose a lightweight runtime library to translate them into FPGA primitive hardware instructions.

First, instructions specific to ReRAM DNN acceleration needs translation, which include ReRAM control instructions, move instructions and “LdWGT”. We save the extra information of “SetCore” and “SetTile” which can support MVM computation scheduling and remove useless “Move” instructions and “SetRoute” instructions. For “LdWGT”, we remap weights and do some necessary data processing.

Second, the translation should be done to adjust mismatched data size since we split the MVM. We load and store a small part of ReRAM input and weight data and get partial sum instead of the final sum when we perform the emulation. For the data transfer instructions, we extend them with some extra fields, e.g., the size and index to move an input subvector and several weight subvectors along output channels at a time as observed in Fig. 3.

TABLE I. INSTRUCTIONS FOR ReRAM DNN ACCELERATOR

Type	Instruction	Description
Data Transfer	Move	Data movement between Tiles.
	Store	Store data to DRAM.
	Load	Load data from DRAM.
	LdWGT	Program data to ReRAM crossbar.
Computation	Pooling	A series of ALU operation to realize pooling operation.
	MVM	Matrix-Vector Multiplication.
	Activation	A series of ALU operation to realize activation operation.
Configurariion	SetCore	Set computation task of the Core.
	SetTile	Set computation task of the Tile.
	SetRoute	Set the way of data routing between Tiles.
Control	Finish	The end of Core/Tile computation.

C. Putting them together

Fig. 4(a) shows the overall flow of the FPGA emulator.

In the first step, the CPU compiles the DL frameworks into separate CPU instructions and accelerator instructions. At this step, some preprocessing, such as weight can be adjusted to take the ReRAM variation model into account as elaborated in the next section. The accelerator instructions will be offloaded to the accelerator after being translated and rescheduled by the runtime software stack.

In the second step, instruction streams are fetched and executed in the accelerator. Fig. 4(b) illustrates the execution of the instruction streams. The emulator gets computing tasks and corresponding data from control instructions and data transfer instructions. Then, computation instructions are executed till the end of the computing tasks and the results are stored to DRAM in the end. By translating and composing different instruction streams, it becomes more flexible to cater for different DNN needs when carrying out the emulation.

In the last step, CPU gets the inference results from DRAM just like them in ReRAM DNN accelerator.

In summary, the FPGA emulation generates instructions by the host CPU, schedules MVM operation process and translates instructions by runtime software and executes them in the FPGA DNN accelerator. The MAC tree in FPGA provides paralleled hardware computing units that accelerate the emulation of ReRAM crossbar. The software stack makes the emulator more flexible via the translation and scheduling.

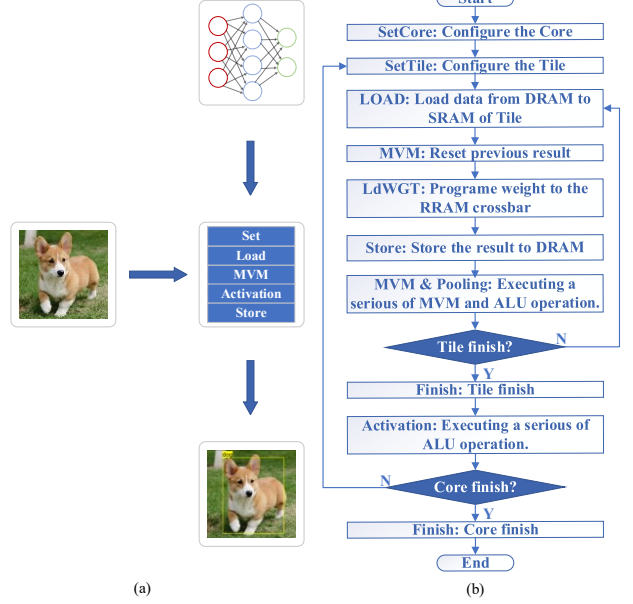


Fig. 4. (a) Overview. (b) How the instruction executes in emulation.

D. Case Study: Emulation with ReRAM variation

The proposed FPGA-based emulation can mimic the ReRAM DNN accelerator in a faster way even for a large-scale DNN. It can be further extended to model ReRAM variation for more faithful ReRAM DNN acceleration. The ReRAM device suffers from various variation in set/reset and read process. In addition, fabrication defects may also cause device resistance stuck at a low or high resistance. These factors may lead to incorrect results on MVM operation that undermines the NN accuracy. As a result, it is necessary to run the ReRAM DNN acceleration with variation model to evaluate the correctness.

In our emulator, we propose to add set/reset variations and device defects inherent to ReRAM crossbars via the LdWGT process by preprocessing weights according to the variation model before loading them into the MAC tree unit. We evaluate their impact on NN accuracy in our experiments. Note that other sources of variations, such as read variations which typically appear as a random telegraph noise (RTN) [14] when weights are accessed in crossbars can also be added into the emulation. It is assumed that R and ΔR are origin resistance and read variation of ReRAM devices, $\Delta R/R$ ranges from 0.1% up to 12% increasing with R [15]. It can be modeled during the MVM process as well.

IV. EXPERIMENT RESULTS

In this section, we first evaluate the acceleration of the proposed FPGA-based emulator which emulates the ReRAM DNN accelerator as shown in Fig. 1. Next, we show that, using the faster emulation, it is possible to evaluate the device

variation impact on DNN accuracy even for large-scale DNN. This capability permits an evaluation of ReRAM accelerator architecture at an early design stage.

The emulator is implemented on Xilinx ZYNQ-7020 FPGA with ARM Cortex-A9 acting as the host CPU, which operates at 100 MHz frequency. Due to the limitation of DSP slices, we implemented a MAC tree to emulate the behavior of a 16×16 ReRAM crossbar and a Tensor ALU for 1×16 vector arithmetic operations. Table II shows the FPGA resources used by the proposed emulator. Note that we have reached 100% utilization on MAC (DSP) resources.

For the software stack, we implemented it based on the open source TVM compiler [16] that can perform optimizations of DL frameworks and generate the instructions for the VTA [17], which is a programmable tensor accelerator for DL workload. We modify the TVM to generate the instructions as shown in Table I. We also split the MVM into a serial of operations via runtime software that can be translated and scheduled on FPGA primitive hardware.

TABLE II. FPGA RESOURCES USAGE

	DSP	BRAM	LUT	FF
Utilization	220	139	37450	38178
Available	220	140	53200	106400
Utilization %	100	99.29	70.39	35.88

A. Speedup over software simulation

To evaluate the speedup capability of the FPGA emulator, we compared it with DNN NeuroSim, a popular software simulator for DNN inference performance for ReRAM. We mapped several DNNs, such as ResNet and VGG-8. We ran the simulator on Intel® Core™ i5-8500 CPU for comparison and turned off the estimation of energy and area to reduce the execution time that matches our emulator.

As Fig.5 shows, the performance of the emulator in terms of execution time (in log scale), gets improved by at least 129.2X for small NNs, and more than 194.7X for larger NNs as ResNet. However, for deeper NNs like ResNet 101, DNN NeuroSim takes several hours to infer an input feature map, which is unacceptable. It was also found that DNN NeuroSim assumes a very simple memory access scheme by reading predefined inputs and weights from intermediate files. Lacking detailed DRAM access modeling, it runs much faster than a full system simulation. As a result, the emulator can gain much superior speedup over software simulation if with more device details, e.g., by taking detailed variation into account.

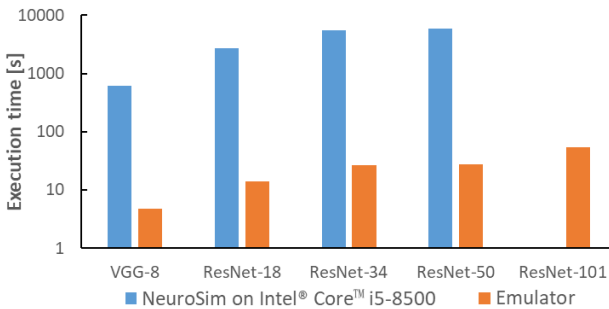


Fig. 5. FPGA emulation speedup over NeuroSim software simulation.

We also developed an in-house C++ cycle-level simulator for the architectural exploration of ReRAM DNN accelerator with detailed DRAM timing models. The FPGA emulator gets 420.8X speedup because of the much longer simulation time that runs on Intel® Core™ i5-8500 for a single convolution layer.

B. A Case Study on DNN Accuracy With ReRAM Variation

We took ResNet as a case study to evaluate the accuracy and the impact of variation in our emulator. We followed the approach in [5], which mainly models the set/reset variation as a Gaussian distribution around the target conductance value [18] and the device effects as a uniform random distribution, respectively. Table III shows the main parameters for the variation model.

TABLE III. PARAMETER SETTING FOR ReRAM VARIATION MODEL

Terms	Value
Normal Variance Write Distribution on Weights	$\sigma/\mu = 0.1$
Low resistance defects occurrence probability	0.5%
High resistance defects occurrence probability	5%

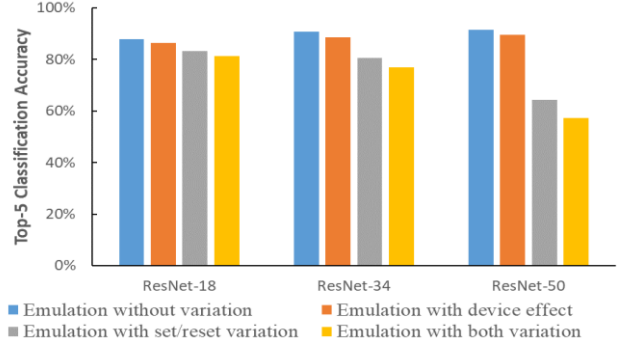


Fig. 6. Impact of variation regarding the Top-5 classification accuracy.

We chose ILSVRC2012 dataset to evaluate the accuracy of the FPGA emulator. Fig. 6 shows the impact of the variation on the accuracy of ResNet Top-5 inference accuracy. For small DNNs, the same classification result was obtained as that on CPU 8-bit quantization software simulation, which validates the correctness of our emulation. For larger DNNs as in the plot, the NN accuracy loss arises from non-ideal devices is 6.43% in ResNet-18, 13.64% in ResNet-34, and 34.04% in ResNet-50, respectively. With larger variation settings, the ReRAM-based MVM calculation deviates more seriously that yields unacceptable classification result in ResNet-50. This justifies the necessity for a fast evaluation at an early design stage for ReRAM DNN accelerator particularly when device variation is taken into account.

V. CONCLUSION

This paper proposes an FPGA emulator, which accelerates the evaluation of general ReRAM DNN accelerator. The faster evaluation time allows more detailed accelerator and device modeling that is important for architecture evaluation. The proposed FPGA emulator caters for different needs from DNNs and ReRAM accelerators by our proposed hardware-software co-design. Therefore, it can provide faster yet flexible hardware emulation for a better ReRAM DNN acceleration design at an early stage.

REFERENCE

- [1] M. Horowitz, "1.1 Computing's energy problem (and what we can do about it)," 2014 IEEE International Solid-State Circuits Conference Digest of Technical Papers (ISSCC), San Francisco, CA, 2014, pp. 10-14, doi: 10.1109/ISSCC.2014.6757323.
- [2] L. Chua, "Memristor-The missing circuit element," in IEEE Transactions on Circuit Theory, vol. 18, no. 5, pp. 507-519, September 1971, doi: 10.1109/TCT.1971.1083337.
- [3] P. O. Vontobel, W. Robinett, P. J. Kuekes, D. R. Stewart, J. Straznicky, and R. S. Williams, "Writing to and reading from a nano-scale crossbar memory based on memristors," Nanotechnology, vol. 20, 2009.
- [4] J. F. Kang et al., "Modeling and design optimization of ReRAM," The 20th Asia and South Pacific Design Automation Conference, Chiba, 2015, pp. 576-581, doi: 10.1109/ASPDAC.2015.7059070.
- [5] T. Luo et al., "An FPGA-Based Hardware Emulator for Neuromorphic Chip With RRAM," in IEEE Transactions on Computer-Aided Design of Integrated Circuits and Systems, vol. 39, no. 2, pp. 438-450, Feb. 2020, doi: 10.1109/TCAD.2018.2889670.
- [6] X. Peng, S. Huang, Y. Luo, X. Sun and S. Yu, "DNN+NeuroSim: An End-to-End Benchmarking Framework for Compute-in-Memory Accelerators with Versatile Device Technologies," 2019 IEEE International Electron Devices Meeting (IEDM), San Francisco, CA, USA, 2019, pp. 32.5.1-32.5.4, doi: 10.1109/IEDM19573.2019.8993491.
- [7] Y. Chen et al., "Diannao family: Energy-efficient hardware accelerators for machine learning," Commun. ACM, 2016.
- [8] A. Shafiee et al., "ISAAC: A Convolutional Neural Network Accelerator with In-Situ Analog Arithmetic in Crossbars," 2016 ACM/IEEE 43rd Annual International Symposium on Computer Architecture (ISCA), Seoul, 2016, pp. 14-26, doi: 10.1109/ISCA.2016.12.
- [9] M. I. Patel and N. Ranganathan, "PANTHER: a parallel neuro-systolic architecture for real-time processing," Proceedings of International Conference on Neural Networks (ICNN'96), Washington, DC, USA, 1996, pp. 1006-1011 vol.2, doi: 10.1109/ICNN.1996.549035.
- [10] A. Ankit et al., "PUMA: A programmable ultra-efficient memristor-based accelerator for machine learning inference," in Proc. 24th Int. Conf. Archit. Support Program. Lang. Operating Syst., 2019, pp. 715-731.
- [11] J. Han, H. Liu, M. Wang, Z. Li and Y. Zhang, "ERA-LSTM: An Efficient ReRAM-Based Architecture for Long Short-Term Memory," in IEEE Transactions on Parallel and Distributed Systems, vol. 31, no. 6, pp. 1328-1342, 1 June 2020, doi: 10.1109/TPDS.2019.2962806.
- [12] Y. Chen et al., "DaDianNao: A Machine-Learning Supercomputer," 2014 47th Annual IEEE/ACM International Symposium on Microarchitecture, Cambridge, 2014, pp. 609-622, doi: 10.1109/MICRO.2014.58.
- [13] J. Ambrosi et al., "Hardware-Software Co-Design for an Analog-Digital Accelerator for Machine Learning," 2018 IEEE International Conference on Rebooting Computing (ICRC), McLean, VA, USA, 2018, pp. 1-13, doi: 10.1109/ICRC.2018.8638612.
- [14] S. Ambrogio, S. Balatti, A. Cubeta, A. Calderoni, N. Ramaswamy and D. Ielmini, "Statistical Fluctuations in HfOx Resistive-Switching Memory: Part II—Random Telegraph Noise," in IEEE Transactions on Electron Devices, vol. 61, no. 8, pp. 2920-2927, Aug. 2014, doi: 10.1109/TED.2014.2330202.
- [15] R. Soni et al., "Probing Cu doped $Ge_{0.3}Se_{0.7}$ based resistance switching memory devices with random telegraph noise," J. Appl. Phys., vol. 107, no. 2, pp. 024517-1-024517-10, Jan. 2010.
- [16] Tianqi Chen, Thierry Moreau, Ziheng Jiang, Lianmin Zheng, Eddie Yan, Meghan Cowan, Haichen Shen, Leyuan Wang, Yuwei Hu, Luis Ceze, Carlos Guestrin, and Arvind Krishnamurthy. Tvm: An automated end-to-end optimizing compiler for deep learning. In Proceedings of the 12th USENIX Conference on Operating Systems Design and Implementation, OSDI18, page 579594, USA, 2018. USENIX Association.
- [17] T. Moreau, T. Chen, L. Vega, J. Roesch, E. Yan, L. Zheng, J. Fromm, Z. Jiang, L. Ceze, C. Guestrin, and A. Krishnamurthy. A hardware-software blueprint for flexible deep learning specialization. IEEE Micro, 39(5):8-16, 2019.
- [18] S. Ambrogio, S. Balatti, A. Cubeta, A. Calderoni, N. Ramaswamy and D. Ielmini, "Statistical Fluctuations in HfOx Resistive-Switching Memory: Part I - Set/Reset Variability," in IEEE Transactions on Electron Devices, vol. 61, no. 8, pp. 2912-2919, Aug. 2014, doi: 10.1109/TED.2014.2330200.